Annexure A

SCENARIO-BASED TASK

QUESTIONS:

1.Scenario: Parsing Table Conflict in LL(1) While designing an LL(1) parser, a student constructs a parsing table and finds multiple entries in a single cell for a non-terminal and input symbol combination. This leads to confusion during parsing. Analyze how the syntax analyzer uses FIRST and FOLLOW sets in this process, identify the reason for the conflict, and explain how it affects deterministic parsing. Conclude by suggesting what decision should be taken to modify the grammar so that it becomes suitable for LL(1) parsing.

2.Scenario: Incorrect Parse Tree Generation A syntax analyzer generates a parse tree for the expression id + id - id, but the evaluation gives incorrect results due to improper grouping of operators. Analyze how the syntax analyzer constructs the parse tree, identify the issue related to associativity, and explain how grammar rules influence the structure of the tree. Finally, discuss what decision should be taken to enforce correct associativity and ensure accurate parsing.

3.Scenario: Invalid Expression Handling A syntax analyzer in a compiler receives the token sequence corresponding to the expression a + * b from the lexical analyzer. Although all tokens are valid individually, the parser fails to generate a parse tree and reports an error. In this situation, analyze how the syntax analyzer identifies the error based on grammar rules, determine the key issue in the token arrangement, and explain how parsing techniques such as predictive or shift-reduce parsing handle such cases. Further, discuss what decision should be taken to improve error detection and recovery while maintaining clarity in parsing.

4. Scenario: Ambiguity in Conditional Statements A syntax analyzer is designed using the grammar S → if E then S | if E then S else S | a. While constructing the parsing table, conflicts arise, and the parser cannot decide which production to apply for certain inputs. In this scenario, explain how the syntax analyzer encounters ambiguity, identify the key issue known as the dangling else problem, and discuss how it impacts parsing. Suggest an appropriate modification or parsing strategy to resolve the ambiguity and ensure correct association of else clauses.

5 Scenario: Operator Precedence Conflict A parser is designed for arithmetic expressions using the grammar E → E + E | E * E | id. While parsing the input id + id * id, the syntax analyzer produces an incorrect parse tree due to ambiguity in operator precedence. Examine how the syntax analyzer processes this input, identify the issue caused by the grammar, and explain how precedence and associativity rules should be applied. Based on this, suggest what changes must be made to the grammar or parsing technique to ensure correct evaluation.

CO2 - AT3

K. Balaji
192324255
B.tech - AI DS

① Parsing table conflict in LL(1)

An LL(1) parser uses the first and follow sets to build a passing table

FIRST(A): The set of terminals that can appear first in strings derived from non-terminal A

Follow(A): The set of Terminals that can appear immediately after A.

When constructing the passing table,

→ if a production starts with terminal in FIRST, it is placed in the corresponding table entry.

→ If a production can derive ε, Then the production is placed under the symbols in Follow

Reason for conflict:

A conflict occurs when two or more productions are entered into the same table cell.

This usually happens because

→ The grammar is ambigious
→ grammar has left recursions
→ two Production have common Prefixes.

Effect on passing:

↳ The parser cannot decide which production to choose.

↳ passing becomes non-deterministic.

↳ The grama is not LL(1).

Decision:

Modify the Grammer by,

Removing left recursion

Applying left factoring

Ensuring every table cell contains only one production.

This makes the grammar suitable for deterministic LL(1) parsing

② Scenario: Incorrect parse tree Generation.

The Syntax analyzer builds a parse tree according to The Grammer rules.

for the expression, Id+Id-Id

The tree should represent the correct operator Grouping.

Issue.

If the Grammer does not specify associative the parser may Group the expression incorrectly.

Example:

Incorrect: Id+(Id-Id)

Correct: (Id+Id)-Id

Role of the Grammer:

Grammer Rule Determine:

→ operator Precedence

→ operator associativity

→ Parse tree structure

If the Grammer is ambigious multiple parse trees are possible.

Decision:

Use Grammar that enforces left associativity.

Example:

$E \rightarrow E + T$

$E \rightarrow E - T$

$E \rightarrow T$

Or remove ambiguity using recursive Grammar this ensures correct parsing and Evaluation.

⑤ Scenario: Invalid Expression handling

Input expression:

a + * b

tokens are individually valid,

Id + * Id

but this order violates grammar Rules

Key Issue:

After the operator +, the grammar expects an operand(Id)

Instead, another operator (*) appears.

Therefore, no valid production can be applied

Parser handling:

Predictive Parser:

→ looks at the parsing table

→ Finds no valid production

→ Reports a Syntax error immediately

Shift reduce parser:

→ Tries Shifting / Reducing

→ Cannot reduce the input into a valid expression

→ Reports a Syntax error.

Decision:

Improve error handling by:

↳ Giving meaningfull error messages.

↳ using panic mode recovery.

↳ Synchronizing with follow sets.

↳ continuing parsing after the error.

This improves compiler usability

④ Scenario: Ambiguity in conditional statements.

Grammer;

S → if E then S

| if E then S else S

| a

Explanation

The parser cannot decide whether an else belongs to the nearest if or an earlier if This is called the hanging Else problem.

Example:

```
if E1 then
    if E2 then
        a
else
    a
```

The else can match either by producing two Parse trees.

Impact
→ Grammar becomes ambigous
→ parsing table contains conflicts.
→ Deterministic parsing is impossible

Decision

Resolve ambiguity by rewriting the grammer using Matching and unmatched statements.

Example:

Matched → If E Then Matched else unmatched / a

unmatched → If E Then S

| If E Then Matched else unmatched

OR use The Rule.

Associate every else with the nearest unmatched If.

this removes Ambiguity

(5) Scenario: operator Precedence conflict:

Grammer:

E → E+E
E → E*E
E → Id

Input:

Id + Id * Id

Explanation:

The grammer is ambigious because it does not specify precedence.

passible parse trees:

1) (Id + Id) * Id
2) Id + (Id * Id)

only the second one is mathematically correct.

Issue:

The parser cannot decide whether + or * should be evaluated first.

Precedence Rules:

- → * has higher precedence than +
- → Both operators are generally left associative

Decision:

Rewrite the Grammar to enforce precedence

Example:

E → E+T/T

T → T*F/F

F → id

Or Specify precedence and associative rules in parser generators such as Yacc/Bison.

This ensures that.

id + id * id

is parsed as

id + (id * id)

giving the correct evalution.

Babyi.k
192324255